

Arquitetura e Implementação de Sistemas de Manutenção Prescritiva com Inteligência Artificial: Guia de Estruturação e Avaliação

A evolução dos paradigmas industriais, transitando da Indústria 4.0 para a Indústria 5.0, exige que a gestão de ativos físicos ultrapasse a manutenção preditiva tradicional, a qual se limita a prever falhas, para adotar a manutenção prescritiva. Esta nova abordagem utiliza a inteligência artificial para recomendar e otimizar ações de reparo específicas e preventivas, mitigando falhas de forma autônoma e contextualmente informada¹. Para estruturar um processo de avaliação prática voltado à contratação de um Desenvolvedor Full Stack Pleno especializado em Inteligência Artificial e Python, é imperativo desenhar um cenário que espelhe a complexidade do mundo real. O desafio proposto requer a ingestão de dados telemétricos provenientes de sensores de vibração em formato JSON, a execução de buscas semânticas em bases documentais técnicas, a inferência por Modelos de Linguagem de Grande Escala (LLMs) para o diagnóstico e a recomendação de soluções, além da persistência do histórico conversacional do usuário e a capacidade de aprender com novos eventos de falha. A estruturação desta avaliação deve mensurar a maturidade do candidato na manipulação de restrições computacionais tangíveis, como a implantação de modelos de linguagem operando localmente em uma placa de vídeo (GPU) com 16 GB de memória VRAM dedicada, bem como a habilidade de orquestrar integrações com plataformas de inferência em nuvem disponíveis gratuitamente. Adicionalmente, o sistema desenvolvido deve exibir resiliência contra alucinações algorítmicas, garantindo que eventos desconhecidos sejam tratados através de esquemas de roteamento lógico estruturado, recusando-se a prescrever soluções para as quais não exista documentação prévia cadastrada.

Fundamentação e Processamento de Dados de Sensores Industriais

O alicerce operacional de um sistema de manutenção prescritiva reside na capacidade de interpretar dados brutos de telemetria e fundi-los com bases de conhecimento textual não estruturadas³. Os dados de entrada estipulados no escopo do projeto originam-se de sensores de vibração acoplados a máquinas rotativas, transmitindo uma carga de dados que contém métricas estatísticas essenciais, como a velocidade RMS (Root Mean Square) em eixos tridimensionais, aceleração de pico, curtose, fator de crista, temperaturas operacionais, rotações por minuto (RPM) e classificações manuais de estado⁵.

Antes de submeter essas matrizes numéricas a um modelo de linguagem, uma arquitetura de software de excelência deve aplicar regras e limiares determinísticos baseados em normas

industriais consagradas. A norma ISO 20816-3 (que atualiza e substitui a norma ISO 10816-3) estabelece as zonas de severidade de vibração para maquinário industrial, classificando a condição operacional das máquinas baseando-se em medições de velocidade RMS de banda larga⁶. Esta filtragem preliminar atua como uma camada de detecção de estado, um componente vital que antecede a análise semântica⁹. A implementação desta etapa demonstra o nível de compreensão do domínio industrial pelo candidato, uma vez que impede o acionamento computacionalmente oneroso de LLMs para estados de vibração que se encontrem dentro das zonas aceitáveis da norma⁷. Apenas anomalias métricas que adentrem os limiares de severidade insatisfatória ou inaceitável justificariam o gatilho da busca semântica em banco de dados e a subsequente geração de relatórios prescritivos. Para que os modelos de linguagem processem esses dados telemétricos com eficácia em tarefas de raciocínio, a técnica arquitetural recomendada é a serialização dos dados estruturados em linguagem natural descritiva¹¹. Frameworks contemporâneos de diagnóstico de falhas em maquinário demonstram que a conversão direta de parâmetros numéricos para sentenças semânticas contextualizadas permite que os LLMs realizem a detecção e classificação de falhas de maneira altamente precisa, utilizando a técnica de demonstração com poucos exemplos prévios¹¹. O candidato deve, portanto, estruturar um módulo de pré-processamento capaz de transformar o formato JSON bruto recebido dos equipamentos em descrições textuais que explicitem a condição do ativo antes de injetar essa informação no modelo gerador.

Arquitetura de Software e Orquestração do Pipeline Backend

Para atender à necessidade de uma solução ponta a ponta que seja executável localmente e demonstrável através de uma interface interativa, a padronização recai sobre a construção de uma arquitetura orientada a microsserviços na linguagem Python¹². O isolamento da lógica de negócios e da inferência de inteligência artificial em uma camada de serviço dedicada assegura a escalabilidade e a estabilidade do sistema frente a volumes contínuos de dados industriais.

A camada de Interface de Programação de Aplicações (API) deve ser desenvolvida utilizando o framework FastAPI, escolhido pela sua implementação nativa de processamento assíncrono e validação automática de dados¹⁴. A inferência de modelos de linguagem é caracterizada por operações de entrada e saída (I/O) de longa duração e pela geração sequencial e cadenciada de marcações lexicais, o que demanda um gerenciamento de recursos não bloqueante. O FastAPI, operando sobre servidores de interface de gateway assíncrono (ASGI), permite o roteamento simultâneo de requisições complexas que englobam a busca em bancos de dados vetoriais, a construção de comandos condicionais e o retorno de respostas em fluxo contínuo sem incorrer na sobrecarga de bloqueio de processos inerente a frameworks web mais antigos¹⁴.

Na camada de serviço, a orquestração do fluxo de trabalho baseia-se na técnica de Geração Aumentada por Recuperação (RAG). Contudo, a simples justaposição de trechos de manuais

técnicos ao comando do modelo é insuficiente para o rigor da manutenção industrial. Requer-se a adoção de um RAG Agêntico, orquestrado através do framework LangGraph, que modela o processo como um autômato de estados finitos compostos por nós cíclicos de decisão¹⁵. Esta arquitetura avalia proativamente a qualidade da informação recuperada, classifica a intenção do usuário ou do sensor, verifica a existência de dados documentais válidos, e toma decisões de roteamento para evitar alucinações baseadas no conhecimento paramétrico generalista do modelo¹⁶.

Bancos de Dados: Persistência NoSQL, Busca Vetorial e Adição de Novas Falhas

O escopo do projeto avaliativo estabelece o uso obrigatório de bancos de dados NoSQL e a implementação de buscas vetoriais para recuperar diretrizes operacionais, mantendo o controle persistente das interações em forma de chat. Bancos de dados documentais são perfeitamente adequados para a persistência do histórico conversacional devido à sua natureza de esquema flexível, sendo capazes de armazenar identificadores de sessão, sequências de mensagens, metadados de telemetria e o estado interno dos agentes de forma nativa¹⁸.

Existem duas abordagens arquiteturais primárias que o candidato pode adotar para satisfazer essas exigências. A primeira é a consolidação através de uma plataforma unificada, como o MongoDB Atlas Vector Search, que integra nativamente o armazenamento documental NoSQL com algoritmos de indexação vetorial de vizinhos mais próximos¹⁸. Esta solução permite que a mesma camada de infraestrutura gerencie o histórico de chat da interface de usuário, as medições temporais recebidas pelas máquinas e os registros vetoriais de manuais de manutenção²¹. A segunda abordagem é o desacoplamento de responsabilidades, onde a memória conversacional é persistida em um banco de dados transacional como o MongoDB ou mesmo o SQLite (através de gerenciadores de memória nativos do LangGraph, como o MongoDBSaver e o SqliteSaver), enquanto as operações de busca semântica em grande escala são delegadas a motores vetoriais altamente otimizados para cálculos de distância de alta dimensionalidade, como o Qdrant ou o ChromaDB¹⁸. O Qdrant apresenta notável vantagem em implantações locais devido ao seu baixo consumo de recursos e sua capacidade robusta de filtragem associada ao vetor, permitindo que a busca documental seja filtrada rigorosamente por metadados, assegurando que o sistema não recupere instruções de manutenção de compressores ao diagnosticar falhas em um rotor elétrico¹⁸.

Um dos requisitos mais críticos e desafiadores delineados pela especificação do projeto é a instrução de que o sistema deve ater-se estritamente aos problemas documentados. Caso o equipamento apresente um padrão anômalo cujas características não possuam correspondência prévia na base documental, o sistema deve abster-se de gerar previsões infundadas, reportando a inexistência de documentação e orientando o usuário a registrar a nova falha⁵. Esta diretiva invalida completamente os dutos tradicionais de RAG que injetam cegamente qualquer resultado aproximado no modelo gerador¹⁶. O sistema deve possuir um nó avaliador que meça a distância euclidiana ou a similaridade por cosseno entre o vetor do

evento anômalo e os vetores de documentação armazenados²⁵. Se o grau de similaridade for inferior a um limiar seguro pré-estabelecido, a rota de geração deve sofrer um curto-circuito, ativando um protocolo de recusa¹⁷.

O processo de "adição de nova falha" materializa-se quando o operador submete a nova documentação ou instrução de resolução para a anomalia recém-descoberta. O candidato deve estruturar um serviço de ingestão dinâmica que capture este novo manual em formato de texto, processe-o através de técnicas de divisão de caracteres para preservar o contexto semântico, utilize o modelo de representação vetorial para codificar os blocos de texto e, por fim, aplique comandos de atualização ou inserção direta no banco vetorial²⁶. Esta técnica garante que a matriz de conhecimento da inteligência artificial seja expandida em tempo real, eliminando qualquer necessidade de retreinamento dos parâmetros internos do modelo de linguagem de grande escala²⁸.

Funcionalidade de Banco de Dados	Componente Tecnológico	Aplicação na Manutenção Prescritiva
Busca Vetorial (ANN)	Qdrant, ChromaDB ou Atlas Vector	Armazenamento de embeddings de relatórios de manutenção, cálculo de similaridade por cosseno.
Filtragem de Metadados	Payload Filtering (Qdrant)	Isolamento da busca por equipamento específico ou linha de montagem, impedindo a mistura de manuais.
Persistência de Memória (Chat)	MongoDB, SQLite (LangGraph Savers)	Retenção do histórico de diálogos, armazenando identificadores de sessão e estados dos agentes lógicos.
Atualização Contínua	Pipeline de Upsert Dinâmico	Processamento e indexação imediata de novos manuais de operação assim que falhas inéditas são catalogadas.

Modelos de Busca Vetorial e Recuperação Semântica

A eficácia técnica de um ecossistema de geração aumentada por recuperação está intrinsecamente condicionada à qualidade do modelo de codificação de texto responsável pela conversão do vocabulário industrial em mapas topológicos de alta dimensão³⁰. A documentação a ser indexada por uma empresa do estado de Santa Catarina será inevitavelmente redigida em português do Brasil, carregada de jargões eletromecânicos, siglas de rolamentos e especificações de tolerância mecânica. Devido a essa complexidade, os modelos de representação vetorial escolhidos devem exibir fortes capacidades multilíngues e de compreensão profunda de contexto⁵.

Em ambientes com limitações de hardware, onde a codificação deve ser realizada localmente na mesma estação que hospeda o modelo generativo, a eficiência de memória dos modelos de *embeddings* é de extrema relevância³⁰. Entre as alternativas de código aberto mais proficientes para esta tarefa, o modelo BGE-M3 (desenvolvido pela Beijing Academy of Artificial Intelligence) destaca-se como a ferramenta mais abrangente. Operando com aproximadamente 568 milhões de parâmetros, o BGE-M3 suporta recuperação densa, esparsa e em múltiplos vetores, possuindo fluência nativa em mais de cem idiomas³³. A geração de vetores de 1024 dimensões por este modelo assegura uma captura fiel das nuances semânticas contidas em extensos relatórios técnicos, lidando sem dificuldades com contextos de até oito mil unidades de fragmentação de palavras³⁴.

Como alternativa de menor pegada computacional, o modelo Nomic-Embed-Text (especialmente em sua segunda iteração com arquitetura de Mistura de Especialistas) oferece excelente desempenho em CPU, gerando vetores compactos de 768 dimensões com uma janela de contexto equivalente, operando de forma altamente eficiente com frações do custo computacional³⁴. Modelos precursores e ainda menores, como o All-MiniLM-L6-v2 pertencente à biblioteca de *Sentence-Transformers*, embora extremamente leves para sistemas restritos a pouca memória RAM, produzem saídas de menor dimensionalidade e janelas de contexto de apenas 512 unidades, correndo o risco de descartar o contexto vital se aplicados à indexação integral de fluxogramas de manutenção industrial³⁰.

A estratégia de fragmentação dos documentos originais deve ser cuidadosamente parametrizada para manter as descrições de tabelas paramétricas e os dados conjunturais dos equipamentos intactos nos blocos submetidos à vetorização, assegurando que o recuperador do sistema alimente o gerador de texto com informações exatas, localizadas e indivisíveis³⁸.

Técnicas de Inferência e Estruturação de Saídas Generativas

Em sistemas aplicados à manufatura autônoma e à gestão da manutenção preditiva e prescritiva, o modelo de inteligência artificial não atua meramente como um conversador assistencial, mas como uma engrenagem de uma linha de produção algorítmica cujos resultados são passados para sistemas de visualização ou controle numérico¹⁴. Portanto, é estritamente proibido que a inferência do LLM devolva suas recomendações em linguagem natural formatada imprevisivelmente ou adornada com gentilezas conversacionais típicas de chatbots comerciais.

Para domar a saída de um modelo generativo e forçá-lo a respeitar formatos predefinidos de sistemas de comunicação, a técnica padrão industrial atual apoia-se em mecanismos de saídas estruturadas, geridas através de validações rigorosas em esquemas de dados. O uso de bibliotecas de validação tipada, como o Pydantic, integrada ao framework LangChain, obriga o modelo de linguagem a renderizar invariavelmente os campos solicitados pelo sistema industrial³⁸. No escopo do desafio prático, a saída do modelo de análise deve obrigatoriamente preencher um objeto JSON contemplando os identificadores `tipo_defeito`, `frequencia_ocorrendia`, `instrucoes_soluciao` e possivelmente os trechos documentais em que o modelo baseou o seu raciocínio⁵.

A arquitetura de *prompts* injetados no sistema também deve incorporar as melhores práticas de restrição geracional. A técnica de Cadeia de Pensamento (Chain-of-Thought) deve ser incorporada dentro do esquema do próprio Pydantic, garantindo que o primeiro campo a ser preenchido pelo modelo no JSON seja uma avaliação sequencial lógica dos sintomas do sensor, seguida pelas conclusões sintéticas da ação de manutenção. Essa obrigatoriedade topológica na construção do JSON atua mitigando respostas imprecisas que ocorrem quando os modelos tentam formular conclusões finais antes de iterar pelas evidências lógicas e mecânicas⁴⁰. A adesão a esses guardiões de formato previne que as funções a jusante no painel de visualização sofram colapsos por desserialização de objetos malformados, consolidando a estabilidade da aplicação⁴².

Viabilidade Computacional para Inferência Local (16GB de VRAM)

A imposição arquitetural de executar o motor de inferência generativa localmente em uma estação de trabalho equipada com 32 GB de memória principal do sistema e restrita a uma placa de vídeo contendo 16 GB de VRAM molda agressivamente as estratégias tecnológicas que o candidato deve adotar⁵. Os requisitos subjacentes para o processamento de modelos de linguagem ditam que cada parâmetro do modelo, se não for comprimido do seu formato original de ponto flutuante de dezesseis bits (FP16), consome por volta de dois bytes de espaço em disco e memória, fazendo com que o mero carregamento estático de um modelo com oito bilhões de parâmetros preencha instantaneamente toda a memória física livre do dispositivo gráfico, deixando margens insignificantes para o gerenciamento de contexto ou armazenamento das representações atencionais temporárias⁴⁴.

Diante deste gargalo de hardware de uso corporativo, a quantização de pesos torna-se o meio incontornável para viabilizar as engrenagens de diagnóstico. Através do uso de quantizações avançadas para representações de números inteiros de menor densidade, tipicamente mapeadas no formato GGUF e executadas a quatro ou cinco bits por parâmetro, é possível alocar modelos cognitivamente densos, reter amplos blocos de memória livre para manter os históricos de telemetria operacionais ativos sem comprometer dramaticamente a precisão matemática da semântica retida pelo modelo durante o pré-treinamento⁴⁵.

A escolha do modelo a ser carregado nessa arquitetura deve equilibrar a capacidade de entender a engenharia mecânica documentada com a agilidade interativa. A família de

modelos da Meta, representada pelo Llama-3.1-8B-Instruct, oferece uma robustez comprovada no segmento linguístico, incluindo pontuações elevadas em exames de proficiência em português e uma versatilidade técnica generalista invejável. Com a quantização no perfil Q4_K_M ou Q8_0, este modelo reduzirá sua pegada alocada de VRAM para entre cinco e oito gigabytes, deixando a outra metade da memória de vídeo inteiramente liberada para suportar contextos longos (arquivos documentais grandes contendo especificações inteiras das máquinas em monitoramento)⁴⁴.

Porém, um perfil que se estabeleceu como o compromisso perfeito para a configuração demandada na especificação do projeto (16 GB de VRAM) são os modelos na classe de quatorze bilhões de parâmetros, especificamente liderados pelo Qwen-2.5-14B-Instruct. Ao ser submetido à quantização de quatro bits, este modelo passará a requisitar algo em torno de nove gigabytes de alocação física da GPU. Esta escolha reserva ainda aproximadamente sete gigabytes para a estruturação do cache das chaves-valor (KV cache)⁴⁶. As arquiteturas desta magnitude ostentam propriedades superlativas nas avaliações dedutivas lógicas, codificação em linguagens de software e resolução metódica de processos lógicos em várias línguas, superando largamente a classe dos modelos menores e estabelecendo-se como uma fundação extremamente poderosa para diagnósticos assertivos na arena prescritiva⁴⁶.

Motores de Inferência: A Arquitetura do Roteador Local

A plataforma designada para hospedar o modelo localmente e lidar com as solicitações provenientes da interface do sistema diferencia sistemas educacionais daqueles forjados para resiliência de produção. As duas abordagens dominantes compreendem plataformas abstraídas de alta conveniência e servidores modulares configurados para alta cadência de requisições.

A interface baseada no utilitário Ollama apoia-se em camadas nativas implementadas em linguagens compiladas de baixo nível para abstrair complexidades logísticas. Exigindo o mínimo de orquestração para executar versões quantizadas com o comando básico do terminal, a ferramenta gerencia organicamente a locação dos tensores da placa gráfica para o processador se sobrecarregada, entregando um ecossistema amigável de instalação limpa⁴⁹. Esta ferramenta mostra-se adequada e confiável para instâncias unificadas, onde o candidato pretenda focar na lógica de negócios e provar o conceito de manutenção sem interrupções por exaustão sistêmica durante cargas normais de interação com poucos utilizadores. Em contraste, para infraestruturas concebidas para sustentar injeção perene de requisições simultâneas – como é natural de uma arquitetura central coletando medições de centenas de sensores ao longo de uma usina –, a adoção do motor vLLM revela o ápice das melhores práticas na engenharia de aprendizado de máquina⁵³. O vLLM materializou-se como um divisor de águas arquitetural ao trazer a técnica PagedAttention para a retenção do estado atencional dos modelos de linguagem. Ele quebra a linearidade exigida pela memória de contexto tradicional alocando os *tokens* processados dinamicamente em fragmentos discretos, erradicando a fragmentação da memória VRAM e facultando processamento unificado simultâneo⁵³. Adicionalmente, o seu agendador de agrupamentos sequenciais otimiza cada ciclo de cálculo da placa gráfica inserindo requisições na fila de processamento sem a

obrigatoriedade de aguardar a finalização dos processos geracionais em andamento, elevando de maneira formidável o volume de *tokens* processados sob carga máxima e eliminando interrupções sistêmicas quando sujeito a instabilidades nos disparos dos sensores industriais⁵³.

Característica	Servidor Ollama	Servidor vLLM
Arquitetura Base	Baseado na biblioteca otimizada llama.cpp. Focado em portabilidade universal em plataformas computacionais mistas.	Estrutura própria em Python, PyTorch e CUDA. Otimizado rigidamente para máxima eficiência em clusters de GPUs dedicadas.
Gerenciamento de Memória	Retenção padrão com descarregamento dinâmico de pesos para a memória sistêmica em cenários de sobrecarga.	Implementação inovadora de <i>PagedAttention</i> , fragmentando vetores atencionais em blocos e contornando exaustão por fragmentação não contígua.
Cadência de Requisições	Altamente adequado para prototipagem sequencial, fluxos isolados, estações singulares e demonstrações pontuais.	Capacidade extrema em escalabilidade concorrente e agendamento assíncrono avançado; ideal para ambientes de múltiplos usuários ou agentes conectados simultaneamente.
Modelos de Retenção (16GB)	Recomendação de 8B a 14B parâmetros (Q4). Retém grande velocidade nas respostas curtas exigidas para diagnóstico RAG.	Extraí toda a largura de banda possível das memórias GDDR para sustentar grandes volumes sem pausas estruturais nos roteadores.

Alternativas de Inferência Externa: APIs Gratuitas e Suas Restrições

A estrutura da avaliação permite inteligentemente que o candidato abdique do uso irrestrito da máquina local de hardware limitante em favor de arquiteturas de nuvem alimentadas por prestadores de plataformas de IA mediante interfaces de aplicação, demonstrando agilidade, resiliência na construção de conectores abstratos e fluidez no manejo de fornecedores de inteligência artificial baseados em servidores hospedados fora da rede fechada⁵. A arquitetura modular do sistema provará seu valor caso consiga intercambiar os endereços base de conexão dos roteadores com alterações simplificadas nas bibliotecas, adotando APIs perfeitamente alinhadas aos protocolos de conexão padronizados na indústria de *software*. O destaque primordial em termos de fornecimento gratuito repousa na Groq, cuja infraestrutura foi edificada com a tecnologia propulsiva das LPU (Language Processing Units) personalizadas, que diferem frontalmente da clássica dependência de matrizes gráficas por adotarem vastas reservas embutidas de memória SRAM ultrarrápida⁵⁷. A plataforma da Groq é capaz de renderizar saídas com velocidades exorbitantes, viabilizando inferências na ordem de trezentos a seiscentos *tokens* processados por segundo para a família de modelos Llama 3.1. A modalidade sem custos disponibilizada (para a realidade tecnológica do ano de 2026) apresenta permissividades abrangentes⁵⁷. O modelo Llama-3.1-8B-Instant suporta o volume formidável de quatorze mil e quatrocentas chamadas diárias a taxas curtas e concentradas de trinta conexões efetuadas por minuto, o que provê recursos amplos para qualquer avaliação teórica, teste ou protótipo funcional denso⁵⁷. Para contextos demandantes de elevado teor explicativo e profunda interpretação, a oferta providencia limites rigorosos mas valiosos da linha superior contendo os vastos conjuntos do GPT-OSS-120B ou do formidável Llama-3.3-70B; porém, a taxa de envio diário destes gigantes reduz para o limiar restrito de mil interações diárias⁵⁶.

Parelhamente ao espectro oferecido, a divisão Google AI Studio abriga acesso não faturado (apenas requisitando subscrição regular) aos eficientes representantes multimodais da série Gemini⁵⁶. Uma força descomunal reside nas arquiteturas Gemini 2.5 Flash e Gemini 3.5 Flash, que dispõem de formidáveis capacidades de deglutição documental ao permitirem o processamento de mais de um milhão de *tokens* atencionais num único envio de contexto, propiciando a retenção orgânica integral dos compêndios da corporação inteira num piscar de olhos e neutralizando preocupações extenuantes de filtragens dimensionais profundas que afetam modelagens compactas⁵⁶. Todavia, os níveis iniciais apresentam contingenciamento restritivo nas faixas contínuas, abrigando no máximo quinze acionamentos por minuto com recargas cíclicas travadas ao marco diário de mil e quinhentas evocações orgânicas completas, forçando o desenvolvimento do arquiteto a adotar pausas forçadas (*backoffs* exponenciais) programáticas sob forte uso simultâneo durante testes massivos⁶².

Avançando rumo a abordagens em nichos emergentes focadas especificamente em escalar os modelos não proprietários além de trilhões de parâmetros, companhias como a SambaNova e a Cerebras alicerçaram provisões com cotas de acesso a modelos de arquiteturas difíceis de se hospedar em locais sem *clusters* corporativos substanciais⁵⁸. A SambaNova expede créditos temporários ou acesso direto que viabilizam o emprego das forças conjuntas gigantescas do conglomerado de Mistura de Especialistas contido na arquitetura DeepSeek V3.2 ou nas classes supremas do Llama 3.1 405B, as quais perfazem métricas incomparáveis de análise

sistemática do linguajar abstruso empregado na elaboração dos guias fabris⁵⁸. Por sua vez, a plataforma baseada nas estruturas colossais da arquitetura *wafer-scale* da Cerebras viabiliza um plano devidamente gratuito e perpetuado permitindo a renderização de impressionantes lotes acumulativos dimensionados a um milhão de *tokens* diários irrestritos sem imposições por número total de faturas submetidas durante o teste estrutural⁵⁶.

Plataforma de Inferência (API)	Modelo de Destaque no Limite Gratuito	Capacidade e Restrição Fundamental (Quota/Limite 2026)	Vantagens Analíticas Principais para a Manutenção Prescritiva
Groq Cloud	Llama-3.1-8B-Instant Llama-3.3-70B-Versatile	Alta vazão RPM. Modelos de 8B: 14.400 requisições diárias. Modelos de 70B: Restrito a 1.000 requisições/dia ⁵⁷ .	Extremamente veloz em tarefas RAG de retorno sintético formatadas via matriz JSON na nuvem.
Google AI Studio	Gemini 2.5 Flash Gemini 3.5 Flash	Quotas menores (1.500 req/dia e 15 RPM). Contexto elástico (1 Milhão de <i>tokens</i> englobáveis na faixa padrão) ⁵⁶ .	Elimina truncamentos semânticos ao absorver enormes lotes teóricos e PDF de engenharia integrais de uma só visada.
Cerebras Cloud	Llama-3.1-8B e 70B Qwen3 Instruct	Processamento massivo limitado rigidamente pelo consumo diário (1.000.000 de <i>tokens</i> contados por sessão dia) ⁵⁶ .	Confere estabilidade a processos avaliativos e fornece integração transparente substituindo as antigas bases sem gargalos logísticos.
SambaNova Cloud	DeepSeek V3.2	Fornecer subvenções	Ativa inferências superlativas de

	Llama-3.1-405B	contidas (\$5 expiráveis). Cadência regulada artificialmente aos limiares de segurança programáticos ⁵⁸ .	lógica corporativa sem a imposição de construir <i>datacenters</i> particulares extensos.
--	----------------	--	--

A maturidade algorítmica e de engenharia de software demonstrada por um desenvolvedor em um projeto deste calibre atinge o ápice quando se projeta configurações resilientes. Implementações utilizando o pacote de carregamento ambiental pydantic-settings revelam o potencial da ferramenta para carregar configurações de ambiente de forma segura e sem intervenção manual⁶⁷. O mecanismo pode facilmente parametrizar um processo em que, por exemplo, ocorra uma quebra sistêmica no provedor via internet ou as cotas mensais gratuitas expirem precipitadamente. O projeto pode reverter organicamente a cadeia da aplicação recarregando silenciosamente a invocação de volta ao ecossistema quantizado nativo alocado no espaço de dezesseis gigabytes locais do microcomputador local mediante roteamento alternativo e salvaguardar a preservação lógica da inferência interligada perante desastres nos domínios remotos interconectados de inteligência descentralizada³⁸.

Interface, Monitoramento Front-end e Interação Assíncrona

A representação final do fluxo de trabalho é materializada por meio de painéis de visualização. Frameworks voltados para a engenharia de dados, como Streamlit e Dash, suprimem a necessidade da construção laboriosa em linguagens de desenvolvimento web de baixo nível em prol de códigos dinâmicos concentrados estritamente no ecossistema da linguagem Python⁷¹.

O desafio demanda um nível considerável de maturidade no controle destas interfaces, pois a apresentação puramente linear de estatísticas estáticas não atende aos requisitos industriais da atualidade⁵. O gerenciamento persistente e consistente do fluxo de interações do operador que supervisiona a automação recai no armazenamento lógico através da ferramenta `st.session_state` associada aos microsserviços do backend descritos anteriormente. Isto faculta manter o ambiente reativo e inalterado e resguarda os longos diálogos da depuração contra interrupções ou apagamentos originados de recarregamentos arbitrários impostos pelos gatilhos cíclicos de renderização visual dessas bibliotecas abstratas⁷¹.

Complementarmente, a incorporação de supervisores de processos passivos, através de pacotes modulares robustos como as abordagens baseadas no sentinela watchdog ou a estratégia flexível proveniente de atualizações temporizadas presentes no `streamlit_autorefresh`, fomenta uma resposta tática do painel visual sem necessitar do impulso manual, injetando proatividade no ecossistema perante aos incessantes disparos JSON gerados paralelamente pelas frotas de sensores industriais e processados pelo API hospedada

separadamente⁷³. Esse elo harmônico em conjunto forja um conjunto observável resiliente onde as detecções, categorizações, desvios e relatórios analíticos dos LLMs despontam ritmadamente sobre os esquemas de supervisão visual da equipe técnica.

Métricas de Avaliação, Observabilidade e Robustez Algorítmica (RAGAS)

Um sistema de geração aumentada que toma decisões autônomas no chão de fábrica e dita normativas processuais prescritivas acarreta responsabilidades e riscos operacionais graves. Uma resposta imprecisa decorrente de alucinações geradas livremente pode levar a acidentes na montagem mecânica, deterioração de peças e prejuízos catastróficos aos negócios⁴³. A auditoria, qualificação quantitativa e aferição probabilística das asserções tecidas pela IA formam a fronteira irrevogável de avaliação entre demonstrações banais e softwares com calibre corporativo implementável em solo catarinense.

Para assegurar o rastreamento, metodologias robustas para a apreciação da qualidade devem ser internalizadas na rotina de avaliação do projeto candidato. Ferramentas instrumentadas especificamente para tais dutos baseados em contexto retido, destacando-se o protocolo padronizado da estrutura RAGAS (Retrieval Augmented Generation Assessment), entregam relatórios sistemáticos livres de predefinições pesadas de viés manual utilizando o próprio núcleo inteligente subjacente de raciocínio profundo como julgador isento⁷⁵. Os principais vetores examinados atestam indubitavelmente a higidez metodológica:

- **Acurácia e Resgate de Contexto (Context Precision / Context Recall):** Essas aferições mensuram rigidamente as porcentagens exatas e as frações posicionais nas quais os documentos relevantes das bibliotecas eletrônicas originais foram extraídas e colocadas fidedignamente à disposição das diretrizes do processamento central sem importar irrelevâncias cruzadas que diluem os conselhos táticos⁷⁷.
- **Fidelidade Algorítmica (Faithfulness):** Representa uma medição elementar crucial punitiva. Avalia com estrita disciplina lógica as correlações para determinar se toda asserção expedida perante ao questionamento formulado advém, inexoravelmente, do conjunto atencional suplementar injetado, não abrigando extrapolações perigosas retiradas do banco genérico desatualizado paramétrico retido dentro dos limiares inertes das matrizes generativas originais, controlando dessa maneira as nocivas presunções não amparadas⁷⁷.
- **Aderência Estruturada à Interrogação (Answer Relevancy):** Audita metodologicamente o texto elaborado avaliando o grau analítico pelo qual toda a porção da resposta prescritiva emitida realmente interliga, sem tangenciar o objetivo, as peculiaridades métricas, mecânicas, logísticas ou estatísticas formuladas pelas anomalias oriundas das medições da rede sensora ativada nos pólos da fabricação rotativa⁷⁶.

Esta habilidade para provar a validade intrínseca do algoritmo perante cenários corrompidos ao injetar componentes defeituosos de observabilidade analítica como o LangSmith reflete, de maneira incontestável e cabal, o elevado pragmatismo na construção da engenharia que

molda a qualificação e sela a contratação de profissionais versados nesta exigente estirpe da ciência de dados¹³.

Consolidação dos Critérios de Avaliação Prática para Entrevista

A síntese que orientará as arguições analíticas por parte do corpo de recrutadores deve segmentar criteriosamente as evidências demonstradas pelo candidato perante as múltiplas vertentes abrigadas neste denso estudo de caso. Os seguintes marcadores qualitativos resumem as expectativas em torno da excelência a serem atestadas em defesa:

1. **Arquitetura, Triagem e Fundamentos Industriais (20% do Escopo Avaliativo):** Será aferida a sobriedade analítica para aplicar tratamentos mecânicos prévios antes da sobreutilização injustificada de processos onerosos. Requer-se a prova inconteste da segregação em JSON ou tabulações de estados benignos classificados originalmente pelos operadores como normais ou inativos, concentrando perfeitamente o processamento complexo apenas para desvios com potencial deletério latente aos sistemas de automação reportados nas faixas métricas insatisfatórias da norma de tolerância contínua estipulada pela documentação internacional ISO equivalente⁵.
2. **Estruturação, Fluxo Lógico e Segurança Operacional Agêntica (30% do Escopo Avaliativo):** O candidato é obrigado a transpor os arranjos simplificados das cadeias geracionais retilíneas em prol das construções modulares encadeadas com mecanismos de fluxos controláveis e determinísticos balizados em engenharias de grafos de estados (LangGraph). Deverão evidenciar a imunização arquitetural profilática com blocos passivos capazes de abortarem prontamente elucubrações indesejáveis caso não sejam identificadas premissas correlatas sólidas e forcem sistematicamente a produção final baseada apenas no preenchimento imutável dos módulos semânticos de bibliotecas normativas robustas (Pydantic), protegendo e padronizando os alertas da interface com chaves JSON irredutíveis ao invés das prosas prolixas¹⁶.
3. **Matriz Vetorial, Atualizações Contínuas e Base Documental Persistida (25% do Escopo Avaliativo):** O postulante à vaga precisa descortinar de modo lúcido a execução exata de metodologias adequadas e precisas nos fatiamentos das diretrizes apostiladas para preservação da continuidade narrativa textual atrelando representações de *embeddings* dimensionais dotadas de fortes matizes multilinguísticas atreladas ao vernáculo sul-americano em soluções compactas ou baseadas em repositórios nativos (como Qdrant ou a intersecção atlasística vetorial no MongoDB). O sistema deverá abarcar irretocavelmente processos dinâmicos para a aquisição superveniente e contínua das novas orientações das equipes na identificação de novidades para suprir as recusas táticas na ausência formativa¹⁸.
4. **Viabilidade Modular, Alocação, Gestão Front-end e Racionalização Custos-Máquina (25% do Escopo Avaliativo):** Consolidada pela exatidão técnica que espelha os arranjos operacionais corporativos desmembrados mediante roteadores FastAPI segregados perfeitamente das estruturas dinâmicas do front-end e de suas respectivas atualizações

cíclicas temporizadas com persistência dialógica, o candidato garantirá excelência expondo opções modulares entre as adequações espremidas na fronteira rígida na casa restrita dos dezesseis gigabytes via processos sofisticados locais de roteamento inteligente assíncrono (vLLM e compactações inteiras em quatro bits dos formidáveis colossos paramétricos reduzidos) contrapondo ou suportando transições indolores para interfaces sem barreiras através dos acessos ultra-performáticos via internet providos por empresas gratuitas com limitações de volumetrias programadas nos arranjos configuracionais das nuvens operacionais expostas na esfera comercial mundial contemporânea (Groq e ecossistema API do Google Gemini)¹⁴.

A exigência do projeto concebido e analisado perpassa de maneira inexorável pelo teste pragmático na condução das restrições sistêmicas limitantes em contraste com as vastidões de dados da ciência algorítmica generativa moderna. Um candidato que ultrapasse a soleira das etapas e seja aprovado manifestar-se-á nitidamente não apenas pela familiaridade das sintaxes estruturadas puras no ecossistema da linguagem Python, mas demonstrará irrevogavelmente seu pertencimento como arquiteto apto na transmutação analítica fundamental, ligando a volatilidade preditiva incerta de matrizes colossais em rotinas e orientações procedimentais inabaláveis e rastreáveis na preservação vital da integridade e sustentabilidade das plantas manufactureiras imersas no seio exigente da era ciberfísica e analítica presente e das complexidades indutivas e operacionais requeridas para a nova conformidade industrial autônoma emergente.

Referências citadas

1. Large Language Model Empowered Deep Reinforcement Learning to Support Pre- scriptive Maintenance - Chalmers ODR, <https://odr.chalmers.se/bitstreams/0e52f4e9-0b68-4609-b66c-cd19736d0a93/download>
2. FEDERAL UNIVERSITY OF SANTA CATARINA TECHNOLOGY CENTER AUTOMATION AND SYSTEMS DEPARTMENT UNDERGRADUATE COURSE IN CONTROL AND AUT, https://repositorio.ufsc.br/bitstream/handle/123456789/256433/TCC_MACHADO_PDFA.pdf?sequence=6&isAllowed=y
3. Large Language Model-Augmented Machine Learning Pipelines for Automated Predictive Intelligence - ResearchGate, https://www.researchgate.net/publication/410857940_Large_Language_Model-Augmented_Machine_Learning_Pipelines_for_Automated_Predictive_Intelligence
4. A Systematic Review of Multi-Sensor Information Fusion for Equipment Fault Diagnosis - University of Liverpool Repository, https://livrepository.liverpool.ac.uk/3189887/1/A_Systematic_Review_of_Multi_Sensor_Information_Fusion_for_Equipment_Fault_Diagnosis.pdf
5. 11 - prova prtica.pdf
6. Vibration Measurement Solution - ICP DAS, https://www.icpdas.com/web/product/catalog/VIB_en.pdf
7. Vibration Sensor PdM: From Sensor Specs to ROI Calculation - Industrial Monitor

Direct,

<https://industrialmonitordirect.com/pl/blogs/knowledgebase/predictive-maintenance-with-vibration-sensors-pdm-field-reference>

8. No 2 (130) 2026 Vol. 33, http://www.bg.pg.gda.pl/pmr/pdf/PMRes_2026_2.pdf
9. ISO 13374 Machine Condition Monitoring: Open Architecture for AI Integration - iFactory AI, <https://ifactoryapp.com/predictive-maintenance/iso-13374-machine-condition-monitoring-open-architecture-ai>
10. In-depth insights into industrial turbo-compressors for smarter maintenance and repair, <https://www.extrica.com/article/25400>
11. Prescriptive Agents based on RAG for Automated Maintenance (PARAM) - arXiv, <https://arxiv.org/pdf/2508.04714>
12. Python Software Development Services - IntexSoft, <https://intexsoft.com/python-development-services/>
13. Building Real-World Agentic AI Apps with LangChain & Streamlit | by Amit Kharche, <https://medium.com/@amitkharche/building-real-world-agentic-ai-apps-with-langchain-streamlit-14ed7f8b330d>
14. Deploying LLMs with FastAPI: Production Guide - Zignuts Technolab, <https://zignuts.com/blog/fastapi-deploy-llms-guide>
15. Built an 11-node LangGraph RAG for Indian legal & financial documents — with PII masking, jailbreak detection, tool calling, and hallucination checks : r/LangChain - Reddit, https://www.reddit.com/r/LangChain/comments/1v9raja/built_an_11node_langgraph_rag_for_indian_legal/
16. LangGraph RAG Agent: Self-Correcting Retrieval Pipeline - machinelearningplus, <https://machinelearningplus.com/gen-ai/langgraph-rag-agent-retrieval-augmented-generation/>
17. Autonomous Legal Research Agent utilizing Flow Engineering. A Multi-Agent RAG framework featuring self-healing research loops, Pydantic-based deterministic guardrails, and real-time observability with LangSmith. Built with LangGraph, Ollama, and ChromaDB. · GitHub, <https://github.com/vamsigudipati/Multi-Agent-Legal-RAG>
18. Best Vector Databases in 2026: A Complete Comparison Guide - Firecrawl, <https://www.firecrawl.dev/blog/best-vector-databases>
19. Top 5 Vector Databases: The Engine Behind Modern AI Industry - Ruh AI Blog, <https://www.ruh.ai/blogs/top-5-vector-databases-engine-behind-modern-ai-industry>
20. llmware - PyPI, <https://pypi.org/project/llmware/>
21. maxc-kudo/ever_works_awesome-mcp-servers - GitHub, https://github.com/maxc-kudo/ever_works_awesome-mcp-servers
22. Persistence - Docs by LangChain, <https://docs.langchain.com/oss/python/langgraph/persistence>
23. Metodologia Orientada a Agentes de Linguagem para Assistência Automotiva: Integrando Engenharia de Prompts em Chatbots Avançad,

- <https://repositorio.ufrn.br/bitstreams/3e6140b6-b943-4b96-a00f-6ddc90f822f6/download>
24. Indexing - Qdrant, <https://qdrant.tech/documentation/manage-data/indexing/>
 25. GCN-Based LSTM Autoencoder with Self-Attention for Bearing Fault Diagnosis - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11314809/>
 26. Vector store integrations - Docs by LangChain, <https://docs.langchain.com/oss/python/integrations/vectorstores>
 27. S3 Ingestion with LangChain - Qdrant, <https://qdrant.tech/documentation/tutorials-build-essentials/data-ingestion-begginners/>
 28. Stop Searching, Start Asking: LangGraph Documentation at Your Fingertips, <https://app.readytensor.ai/publications/stop-searching-start-asking-langgraph-docs-at-your-fingertips-H2rbE1JDof4y>
 29. Turn Any Documentation Site Into an AI Agent with LangGraph and Firecrawl, <https://www.firecrawl.dev/blog/build-documentation-agent-langgraph-firecrawl>
 30. Embedding Stack for AI-Powered Applications: Architecture Patterns, <https://embeddingstack.com/embedding-stack-for-ai-applications/>
 31. A Ransomware Detection Approach Based on LLM Embedding and Ensemble Learning, <https://www.techscience.com/cmcs/v87n1/66096/html>
 32. Schema Retrieval with Embeddings and Vector Stores Using Retrieval-Augmented Generation and LLM-Based SQL Query Generation - MDPI, <https://www.mdpi.com/2076-3417/16/2/586>
 33. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation | Request PDF - ResearchGate, https://www.researchgate.net/publication/384220744_M3-Embedding_Multi-Linguality_Multi-Functionality_Multi-Granularity_Text_Embeddings_Through_Self-Knowledge_Distillation
 34. The Best Open-Source Embedding Models in 2026 - BentoML, <https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models>
 35. Embedding models comparison: OpenAI, Google, Qwen, Nomic, Jina, BAAI - SurrealDB, <https://surrealdb.com/blog/embedding-models-comparison>
 36. Best Embedding Models for RAG (2026): Ranked by MTEB Score, Cost, and Self-Hosting, <https://www.prem.ai/blog/best-embedding-models-for-rag-2026-ranked-by-mteb-score-cost-and-self-hosting/>
 37. Leveraging Large Language Models for Sensor Data Retrieval - MDPI, <https://www.mdpi.com/2076-3417/14/6/2506>
 38. LangChain Best Practices - Swarnendu De, <https://www.swarnendu.de/blog/langchain-best-practices/>
 39. LLM RAG Tutorial: How to Build a Reliable Retrieval Pipeline - LaunchDarkly, <https://launchdarkly.com/blog/llm-rag-tutorial/>
 40. What is Prompt Engineering? Guide for 2026 - Future AGI, <https://futureagi.com/blog/what-is-prompt-engineering/>
 41. Structured Output | Strands Agents, <https://strandsagents.com/docs/user-guide/concepts/agents/structured-output/>

42. Model IO - KodeKloud Docs,
<https://notes.kodekloud.com/docs/LangChain/Key-Components-of-LangChain/Model-IO/page>
43. AI Guardrails — Production LLM Safety Guide (2026) - MyEngineeringPath,
<https://myengineeringpath.dev/genai-engineer/ai-guardrails/>
44. meta-llama/Llama-3.1-8B-Instruct - Hugging Face,
<https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>
45. Which Quantization Should I Use? A Unified Evaluation of Llama.cpp Quantization on Llama-3.1-8B-Instruct - arXiv, <https://arxiv.org/pdf/2601.14277>
46. Need some LLM model recommendations on RTX 3060 12GB and 16GB RAM - Reddit,
https://www.reddit.com/r/LocalLLaMA/comments/1rv99g7/need_some_llm_model_recommendations_on_rtx_3060/
47. I benchmarked 8 LLM models on a CPU-only Mini PC (i9-12900H, 32GB DDR5) running on Proxmox — here are the results - Reddit,
https://www.reddit.com/r/LocalLLM/comments/1tdwbai/i_benchmarked_8_llm_models_on_a_cpuonly_mini_pc/
48. richardyoung/llama-3.1-8b-instruct-abliterated - Ollama,
<https://ollama.com/richardyoung/llama-3.1-8b-instruct-abliterated:latest>
49. Running Ollama on RTX 5070: Fast Local LLMs on 12 GB GDDR7 Blackwell - GIGAGPU, <https://gigagpu.com/rtx-5070-ollama/>
50. What models you guys running on 8GB? 16GB VRAM? 24GB? 32GB? 48GB? - Reddit,
https://www.reddit.com/r/LocalLLaMA/comments/1u3c8q4/what_models_you_guys_running_on_8gb_16gb_vram/
51. A Systematic Evaluation of On-Device LLMs: Quantization, Performance, and Resources, <https://arxiv.org/html/2505.15030v5>
52. Step-By-Step LLM Serving Guide for Production AI Systems - Inference.net,
<https://inference.net/content/llm-serving-guide/>
53. LLM Inference Engine Showdown: vLLM vs Ollama vs TGI | Build with Matija,
<https://www.buildwithmatija.com/blog/vllm-vs-ollama-vs-tgi-choose-llm-inference-engine>
54. vLLM for beginners: The Fundamentals - Cloudthrill,
<https://cloudthrill.ca/what-is-vllm>
55. Benchmarking Ollama and vLLM for Concurrent LLM Serving: A Multi-Scenario Evaluation of Performance and Scalability - MDPI,
<https://www.mdpi.com/2076-3417/16/11/5435>
56. The AI Engineers' Guide to The Top 10 Genuinely Free AI Inference Providers in 2026,
<https://thomascherickal.com/2026/06/24/the-ai-engineers-guide-to-the-top-10-genuinely-free-ai-inference-providers-in-2026/>
57. Groq API Guide 2026: Setup, Pricing, Endpoints & Code Examples - Ampcome,
<https://www.ampcome.com/post/how-to-use-groq-api-the-comprehensive-guide-you-need>
58. AI Inference Providers 2026: Free Tier Deep-Dive for CTOs and Data Teams -

- Vadzim Belski,
https://belski.me/blog/ai_inference_providers_2026_free_tier_deep_dive/
59. How to Use Groq's FREE API in Your Streamlit App - Bitdoze,
<https://www.bitdoze.com/groq-api-mistral-streamlit/>
 60. Introduction - Free LLM API Resources,
<https://cheahjs-free-llm-api-resources.mintlify.app/introduction>
 61. Gemini 1.5 Flash API: Rate Limits & Migration Guide - Rapid Developers,
<https://www.rapidevelopers.com/ai-api-limits-performance-matrix/gemini-1-5-flash>
 62. Gemini API Free Tier 2026: Limits, Quotas, and More - PE Collective,
<https://pecollective.com/tools/gemini-free-tier-guide/>
 63. Gemini API Pricing May 2026: 3.5 Flash, 3.1 Pro, 2.5 Lite - Metacto,
<https://www.metacto.com/blogs/the-true-cost-of-google-gemini-a-guide-to-api-pricing-and-integration>
 64. Gemini API Rate Limits Explained: Complete 2026 Guide with All Tiers,
<https://www.aifreeapi.com/en/posts/gemini-api-rate-limit-explained>
 65. SarmaLink-AI, Technical Whitepaper,
<https://www.sarmalinux.com/products/sarmalink-ai/whitepaper>
 66. Cerebras Launches the World's Fastest AI Inference,
<https://www.cerebras.ai/press-release/cerebras-launches-the-worlds-fastest-ai-inference>
 67. Settings | Pydantic Docs, <https://pydantic.dev/docs/validation/1.10/usage/settings/>
 68. Pydantic settings,
https://pydantic.dev/docs/validation/2.0/usage/pydantic_settings/
 69. Pydantic Environment Variables: A Comprehensive Guide - Orchestra,
<https://www.getorchestra.io/guides/pydantic-environment-variables-a-comprehensive-guide>
 70. Llama 3.1 8B Instruct - API Pricing & Benchmarks | OpenRouter,
<https://openrouter.ai/meta-llama/llama-3.1-8b-instruct>
 71. Streamlit UI: Framework for Building Interactive AI Apps - ThirdEye Data,
<https://thirdeyedata.ai/technologies/streamlit-ui>
 72. Python for Engineering Students: Top Projects to Build in 2026 - WiredWhite,
<https://wiredwhite.com/python-projects-for-engineering-students/>
 73. Real-Time Dashboards with FastAPI and Streamlit watchdog and Streamlit_autorefresh | by Amudhapriya | Medium,
<https://medium.com/@ammusambanthar/real-time-dashboards-with-fastapi-and-streamlit-watchdog-and-streamlit-autorefresh-ce43dad9664>
 74. PHMForge: Evaluating LLM Agents on Industrial Prognostics through MCP-Native, Algorithm-Grounded Tools - arXiv, <https://arxiv.org/html/2604.01532v2>
 75. RAGAs: Automated Evaluation of Retrieval Augmented Generation | Request PDF,
https://www.researchgate.net/publication/393020278_RAGAs_Automated_Evaluation_of_Retrieval_Augmented_Generation
 76. Procedure Knowledge Extraction using Agentic RAG | by Vincent C | MITB For All | Medium,
<https://medium.com/mitb-for-all/procedure-knowledge-extraction-using-agentic>

[-rag-fdf93e028de0](#)

77. A Step-by-Step RAG Evaluation Process & Key Metrics Explained - Openxcell,
<https://www.openxcell.com/blog/rag-evaluation/>

78. RAG Evaluation Framework: Accuracy, Grounding, Hallucinations - AlVeda,
<https://aiveda.io/blog/rag-evaluation-framework-accuracy-grounding-hallucinations/>